Documentación Frontend — Plataforma de Clubs de Lectura

ÍNDICE

1. Introducción y tecnologías
2. Estructura del proyecto
3. Sistema de diseño y estilos
4. Gestión de estado y contextos
5. Sistema de enrutamiento
6. Módulos de API
7. Componentes reutilizables
8. Páginas y flujos de usuario
9. Patrones y convenciones
10. Configuración y despliegue

---

1. Introducción y tecnologías

El frontend de la plataforma es una **aplicación de página única (SPA)** construida con React 18 y TypeScript. Se comunica exclusivamente con el backend Symfony a través de la API REST bajo el prefijo `/api`, sin servidor de renderizado propio. El HTML resultante de la compilación con Vite se despliega como archivos estáticos servidos por Nginx.

Stack tecnológico

Tecnología	Versión	Función
React	18.3.1	Framework de componentes reactivos
TypeScript	5.5.3	Tipado estático en tiempo de desarrollo
React Router DOM	6.26.1	Enrutamiento en el cliente (SPA)
Vite	5.4.1	Bundler y servidor de desarrollo con HMR
Lucide React	1.7.0	Biblioteca de iconos SVG
CSS Custom Properties	—	Sistema de tokens de diseño (sin frameworks externos)

La decisión de no utilizar ningún framework CSS externo (Bootstrap, Tailwind, etc.) responde a una exigencia del proyecto: todo el sistema visual está construido a medida con CSS Custom Properties definidas en `tokens.css`, lo que garantiza control total sobre el diseño y elimina dependencias de terceros en la capa visual.

Integración con el backend

Durante el desarrollo, Vite actúa como proxy: las peticiones a `/api/*` y `/uploads/*` se redirigen automáticamente a `http://127.0.0.1:8000` (el servidor Symfony local). En producción, Nginx hace el mismo trabajo de enrutamiento, de modo que el frontend compilado nunca necesita conocer la dirección real del backend.

La autenticación se basa en **cookies de sesión**: todas las llamadas HTTP incluyen `credentials: 'include'` para enviar automáticamente la cookie `PHPSESSID` que Symfony gestiona. No se utilizan tokens JWT ni cabeceras de

autorización.

---

2. Estructura del proyecto

```
frontend/
├── index.html                  # Punto de entrada HTML (carga el módulo main.tsx)
├── vite.config.ts              # Configuración del bundler y proxy de desarrollo
├── tsconfig.json               # Configuración TypeScript (modo estricto)
├── package.json                # Dependencias y scripts npm
└── src/
    ├── main.tsx                # Punto de entrada React: monta <App /> en #root
    ├── index.css               # Importaciones de todos los módulos CSS
    ├── App.tsx                 # Árbol de providers y definición de rutas
    ├── api/                    # Módulos de llamadas HTTP al backend (12 archivos)
    │   ├── client.ts           # Funciones base: apiFetch, apiFormData
    │   ├── auth.ts             # Login, logout, registro, sesión actual
    │   ├── books.ts            # Búsqueda e importación de libros
    │   ├── reviews.ts          # Reseñas y valoraciones de libros
    │   ├── shelves.ts          # Estanterías y libros en ellas
    │   ├── readingProgress.ts  # Seguimiento activo de lectura
    │   ├── clubs.ts            # Clubs, miembros, solicitudes y libro del mes
    │   ├── chats.ts            # Hilos de chat y mensajes
    │   ├── posts.ts            # Publicaciones, likes y comentarios
    │   ├── profile.ts          # Perfil propio, avatar, privacidad y contraseña
    │   ├── users.ts            # Búsqueda de usuarios
    │   └── admin.ts            # Panel de administración
    ├── context/
    │   └── AuthContext.tsx     # Proveedor de autenticación global
    ├── components/             # Componentes reutilizables (6 archivos)
    │   ├── Navbar.tsx          # Barra de navegación con notificaciones
    │   ├── Footer.tsx          # Pie de página
    │   ├── PrivateRoute.tsx    # Guardia de rutas protegidas
    │   ├── Toast.tsx           # Sistema de notificaciones toast
    │   ├── PostCard.tsx        # Tarjeta de publicación social
    │   └── Spinner.tsx         # Indicador de carga
    ├── pages/                  # Componentes de página, uno por ruta (12 archivos)
    │   ├── HomePage.tsx
    │   ├── LoginPage.tsx
    │   ├── RegisterPage.tsx
    │   ├── BooksPage.tsx
    │   ├── BookDetailPage.tsx
    │   ├── ClubsPage.tsx
    │   ├── ClubDetailPage.tsx
    │   ├── ShelvesPage.tsx
    │   ├── ProfilePage.tsx
    │   ├── PublicProfilePage.tsx
    │   ├── UsersPage.tsx
    │   └── AdminPage.tsx
    └── styles/                 # Módulos CSS organizados por funcionalidad (9 archivos)
        ├── tokens.css          # Variables del sistema de diseño
        ├── layout.css          # Navbar, footer, banners, grids de página
```

```
        ├── components.css      # Botones, formularios, alertas, modales, badges
        ├── auth.css            # Login y registro
        ├── home.css            # Hero, features, stats
        ├── books.css           # Búsqueda, detalle de libro, ShelfDrawer
        ├── clubs.css           # Cards de club, detalle, chats, modales
        ├── shelves.css         # Estanterías, ReadingTracker, progress bars
        └── profile.css         # Perfil propio, perfil público, post gallery
```

Punto de entrada y árbol de providers

`main.tsx` crea la raíz React y monta `<App />`. `App.tsx` define el árbol de providers que envuelve toda la aplicación, de exterior a interior:

```
<ToastProvider>          ← gestiona la cola de notificaciones toast
  <AuthProvider>         ← gestiona la sesión del usuario
    <BrowserRouter>      ← habilita el enrutamiento en cliente
      <Navbar />
      <main>
        <Routes />       ← 12 rutas definidas con react-router-dom
      </main>
      <Footer />
    </BrowserRouter>
  </AuthProvider>
</ToastProvider>
```

---

3. Sistema de diseño y estilos

Todo el sistema visual está construido sobre **CSS Custom Properties** (variables CSS) sin ningún framework externo. `tokens.css` es el archivo central del que derivan todos los demás módulos CSS.

3.1 Paleta de colores

La paleta sigue un esquema tricolor con tema oscuro:

Color primario — Violeta

```
--color-primary:       #7c3aed   /* Violeta base */
--color-primary-hover: #8b4cf6   /* Hover / enfatizado */
--color-primary-dark:  #3f008e   /* Fondo activo / pressed */
--color-accent:        #d2bbff   /* Acento claro */
--color-accent-2:      #c0a8ff   /* Acento claro secundario */
```

Color secundario — Cian

```
--color-cyan:          #c0c1ff
--color-cyan-2:        #a8aaf0
```

Color de peligro — Rosa/Rojo

```
--color-rose:          #ffb4ab
--color-rose-2:        #ff8a80
--color-danger:        #ffb4ab
--color-danger-bg:     rgba(147, 0, 10, 0.18)
--color-danger-border: rgba(255, 180, 171, 0.30)
```

Color de éxito

```
--color-success: #6ee7b7
```

Fondos y superficies

```
--color-bg:             #15121b   /* Fondo global de página */
--color-surface:        #221e28   /* Tarjetas, paneles, modales */
--color-surface-tinted: #2c2833   /* Superficies anidadas */
--color-surface-glass:  rgba(34, 30, 40, 0.60)   /* Glassmorphism */
```

Texto

```
--color-text:       #e8dfee   /* Texto principal */
--color-text-muted: #ccc3d8   /* Texto secundario / descriptivo */
--color-text-light: #958da1   /* Texto terciario / placeholders */
```

Bordes

```
--color-border:       rgba(255, 255, 255, 0.12)
--color-border-light: rgba(255, 255, 255, 0.07)
```

3.2 Gradientes

```
--gradient-primary: linear-gradient(135deg, #7c3aed 0%, #a855f7 100%)
--gradient-purple:  linear-gradient(135deg, #7c3aed 0%, #d2bbff 100%)
--gradient-cyan:    linear-gradient(135deg, #5254df 0%, #c0c1ff 100%)
--gradient-rose:    linear-gradient(135deg, #93000a 0%, #ffb4ab 100%)
```

3.3 Sombras

```
--shadow-sm: 0 1px 3px rgba(0,0,0,.45), 0 1px 2px rgba(0,0,0,.30)
--shadow-md: 0 4px 16px rgba(0,0,0,.55), 0 2px 6px rgba(0,0,0,.40)
--shadow-lg: 0 8px 32px rgba(0,0,0,.60), 0 4px 12px rgba(0,0,0,.40)
--shadow-xl: 0 20px 60px rgba(0,0,0,.70), 0 8px 24px rgba(0,0,0,.50)
--shadow-colored: 0 4px 20px rgba(124,58,237,.35), 0 2px 8px rgba(124,58,237,.20)
```

3.4 Tipografía

```
--font-sans:    'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif
--font-display: 'Newsreader', Georgia, 'Times New Roman', serif
```

Inter se usa para todo el texto funcional: etiquetas, botones, formularios, contenido de lectura.
Newsreader (serif) se reserva para los titulares del hero y secciones destacadas, aportando un carácter editorial coherente con la temática literaria de la plataforma.

Ambas fuentes se cargan desde Google Fonts con `font-display: swap` para evitar parpadeo de texto invisible (FOIT) durante la carga.

3.5 Radios de borde y espaciado

```
--radius-sm:  6px
--radius-md:  10px
--radius-lg:  16px
--radius-xl:  24px
--radius-2xl: 32px
--radius-pill: 999px   /* Badges, chips, pills */
```

3.6 Transiciones y animaciones

```
--transition:        180ms cubic-bezier(0.4, 0, 0.2, 1)   /* Interacciones estándar */
--transition-slow:   320ms cubic-bezier(0.4, 0, 0.2, 1)   /* Aperturas de panel */
--transition-bounce: 400ms cubic-bezier(0.34, 1.56, 0.64, 1) /* Elementos que "saltan" */
```

Keyframes definidos en `tokens.css`:

Animación	Uso
`spin`	Spinner de carga (rotación 360°)
`fade-in-up`	Entrada de tarjetas y secciones (opacidad + traslación Y)
`scale-in`	Apertura de modales (escala desde 0.95 a 1)
`slide-in-right`	Drawer lateral (traslación X desde la derecha)
`slideUp`	Toasts (aparición desde abajo)
`float`	Elemento flotante del hero con ligera rotación
`pulse-ring`	Anillo de pulso expandiéndose para notificaciones
`btn-shimmer`	Efecto de brillo sobre botones primarios
`gradient-x`	Gradiente animado en elementos de fondo

3.7 Responsividad

Los estilos aplican breakpoints en los siguientes anchos de ventana:

Breakpoint	Cambios principales
900px	El layout de dos paneles (estanterías, clubs) colapsa a uno
768px	Navbar cambia a menú hamburguesa; rejillas de 3 columnas pasan a 2
640px	Rejillas de 2 columnas pasan a 1
560px	Hero, estadísticas y cabecera de perfil se reorganizan
480px	Ajustes tipográficos y de padding en móviles pequeños

---

4. Gestión de estado y contextos

4.1 AuthContext

`context/AuthContext.tsx` es el único proveedor de estado global de la aplicación. Expone el usuario autenticado y las operaciones de sesión a todos los componentes del árbol mediante el hook `useAuth()`.

Interfaz del contexto:

```
interface AuthContextType {
  user: AuthUser | null   // null si no hay sesión activa
  loading: boolean        // true durante la verificación inicial
  login: (email: string, password: string) => Promise<void>
  logout: () => Promise<void>
  refresh: () => Promise<void>
}

interface AuthUser {
  id: number
  email: string
  displayName: string | null
  avatar: string | null
  roles: string[]         // p. ej. ['ROLE_USER', 'ROLE_ADMIN']
}
```

Ciclo de vida:

1. Al montar `<AuthProvider>`, se llama a `refresh()` que hace `GET /api/auth/me`.
2. Si hay sesión activa, el servidor responde con los datos del usuario → `user` se actualiza.
3. Si no hay sesión, el servidor responde 401 → `user` queda en `null` (el error se captura silenciosamente).
4. `loading` pasa a `false` en cualquier caso, permitiendo que `<PrivateRoute>` tome decisiones de redirección.

Operaciones:

- `login(email, password)` → `POST /api/login` → actualiza `user` con la respuesta.
- `logout()` → `POST /api/logout` → pone `user` a `null`.
- `refresh()` → `GET /api/auth/me` → re-sincroniza el estado con la sesión del servidor.

4.2 ToastContext

`components/Toast.tsx` exporta `<ToastProvider>` y el hook `useToast()`. Es la segunda fuente de estado global y gestiona la cola de notificaciones no bloqueantes.

```
type ToastType = 'success' | 'error' | 'info'

// Uso desde cualquier componente:
const { toast } = useToast()
toast('Estantería creada', 'success')
toast('Error al guardar', 'error')
toast('Ya eres miembro', 'info')
```

Cada notificación se autodismiss a los 3 500 ms. Los toasts se apilan en la esquina inferior derecha con `aria-live="polite"` para accesibilidad.

4.3 Estado local

El resto del estado de la aplicación es **local al componente** que lo necesita, gestionado con `useState` y `useEffect` de React. No se usa ninguna biblioteca de gestión de estado global adicional (Redux, Zustand, etc.). Esta decisión es adecuada dado el alcance de la aplicación: la mayoría de los datos son específicos de la página activa y no necesitan compartirse entre rutas.

---

5. Sistema de enrutamiento

`App.tsx` define las rutas de la aplicación usando `<Routes>` y `<Route>` de React Router DOM 6.

5.1 Tabla de rutas

Ruta	Componente	Protección	Descripción
`/`	`HomePage`	Pública	Inicio con hero y feed social
`/login`	`LoginPage`	Pública	Inicio de sesión
`/register`	`RegisterPage`	Pública	Registro de cuenta nueva
`/books`	`BooksPage`	Pública	Búsqueda de libros
`/books/:externalId`	`BookDetailPage`	Pública	Detalle de libro, reseñas y valoraciones
`/clubs`	`ClubsPage`	Pública	Lista de clubs de lectura
`/clubs/:id`	`ClubDetailPage`	Pública	Chats, miembros y gestión del club
`/users`	`UsersPage`	Pública	Búsqueda de lectores
`/users/:id`	`PublicProfilePage`	Pública	Perfil público de otro usuario
`/admin`	`AdminPage`	Rol admin	Panel de administración
`/shelves`	`ShelvesPage`	`<PrivateRoute>`	Estanterías personales
`/profile`	`ProfilePage`	`<PrivateRoute>`	Configuración de cuenta y publicaciones

5.2 PrivateRoute

El componente `<PrivateRoute>` protege las rutas `/shelves` y `/profile`. Su lógica:

1. Si `loading` es `true` (aún se está verificando la sesión): muestra `<Spinner>` centrado.
2. Si `user` es `null` (no autenticado): redirige a `/login` guardando la URL de origen en `location.state.from`.
3. Si hay usuario: renderiza `children`.

Tras un login exitoso, `LoginPage` lee `location.state.from` y redirige al usuario a la URL que intentaba visitar originalmente.

5.3 AdminPage

La ruta `/admin` no usa `<PrivateRoute>` porque su protección es por **rol**, no solo por autenticación. `AdminPage` verifica `user?.roles?.includes('ROLE_ADMIN')` al montarse y ejecuta `useNavigate('/')` inmediatamente si la condición no se cumple. Esto permite que un usuario autenticado sin rol de admin no acceda al panel aunque conozca la URL.

---

6. Módulos de API

6.1 Cliente base — `api/client.ts`

Todos los módulos de API utilizan dos funciones base exportadas desde `client.ts`:

```
// Petición JSON estándar
apiFetch<T>(path: string, method?: string, body?: unknown): Promise<T>

// Petición multipart/form-data (subida de archivos)
apiFormData<T>(path: string, formData: FormData): Promise<T>
```

Comportamiento común a ambas:

- La URL se construye como `/api` + `path`.
- Se incluye `credentials: 'include'` en todas las peticiones para enviar la cookie de sesión.
- Si el servidor responde con un código HTTP no-ok (4xx, 5xx), se extrae el campo `error` del JSON y se lanza `new Error(data.error)`. Los componentes capturan este error en sus bloques `catch` y lo muestran al usuario.
- Las respuestas 204 (No Content) se devuelven como `null`.

6.2 Módulo de autenticación — `api/auth.ts`

Función	Método	Endpoint	Descripción
`authApi.me()`	GET	`/auth/me`	Devuelve el usuario de sesión
`authApi.login(email, password)`	POST	`/login`	Inicia sesión (ruta Symfony Security, fuera del prefijo `/auth`)
`authApi.register(email, password, name)`	POST	`/auth/register`	Crea cuenta nueva
`authApi.logout()`	POST	`/auth/logout`	Cierra sesión

6.3 Módulo de libros — `api/books.ts`

Función	Método	Endpoint	Descripción
`booksApi.search(params)`	GET	`/books/search`	Búsqueda con q/title/author/page/limit
`booksApi.get(externalId)`	GET	`/books/{externalId}`	Detalle de un libro concreto

Tipo `Book`: `id`, `externalId`, `title`, `subtitle`, `authors[]`, `thumbnail`, `coverUrl`, `description`, `publisher`, `publishedDate`, `language`, `pageCount`, `categories[]`, `isbn10`, `isbn13`, `averageRating`, `ratingsCount`, `previewLink`, `infoLink`.

6.4 Módulo de reseñas — `api/reviews.ts`

Función	Método	Endpoint	Descripción
`reviewsApi.list(externalId)`	GET	`/books/{externalId}/reviews`	Lista reseñas y estadísticas
`reviewsApi.upsert(externalId, rating, content)`	POST	`/books/{externalId}/reviews`	Crea o actualiza la reseña propia
`reviewsApi.delete(externalId)`	DELETE	`/books/{externalId}/reviews`	Elimina la reseña propia

Tipos: `Review` (`id`, `rating`, `content`, `createdAt`, `user`), `ReviewStats` (`average`, `count`).

6.5 Módulo de estanterías — `api/shelves.ts`

Función	Método	Endpoint	Descripción
`shelvesApi.list()`	GET	`/shelves`	Lista todas las estanterías propias
`shelvesApi.create(name)`	POST	`/shelves`	Crea una estantería
`shelvesApi.delete(id)`	DELETE	`/shelves/{id}`	Elimina una estantería
`shelvesApi.books(id)`	GET	`/shelves/{id}/books`	Libros de una estantería
`shelvesApi.addBook(shelfId, externalId, status)`	POST	`/shelves/{shelfId}/books`	Añade un libro a la estantería
`shelvesApi.updateStatus(shelfId, bookId, status)`	PATCH	`/shelves/{shelfId}/books/{bookId}`	Cambia estado de lectura
`shelvesApi.removeBook(shelfId, bookId)`	DELETE	`/shelves/{shelfId}/books/{bookId}`	Quita un libro de la estantería
`shelvesApi.moveBook(srcId, bookId, dstId)`	POST	`/shelves/{srcId}/books/{bookId}/move`	Mueve libro a otra estantería

Tipo `ReadingStatus`**:** `'want_to_read' | 'reading' | 'read'`

6.6 Módulo de progreso de lectura — `api/readingProgress.ts`

Función	Método	Endpoint	Descripción
`readingProgressApi.list()`	GET	`/reading-progress`	Lista todos los seguimientos activos
`readingProgressApi.add(externalId, mode, totalPages?)`	POST	`/reading-progress`	Inicia seguimiento de un libro
`readingProgressApi.update(id, patch)`	PATCH	`/reading-progress/{id}`	Actualiza progreso (página o %)
`readingProgressApi.delete(id)`	DELETE	`/reading-progress/{id}`	Elimina un seguimiento

Tipo `ReadingProgressItem`**:** `id`, `mode` (`pages|percent`), `currentPage`, `totalPages`, `percent`, `computed` (0-100), `startedAt`, `updatedAt`, `book`.

6.7 Módulo de clubs — `api/clubs.ts`

Función	Método	Endpoint	Des
`clubsApi.list()`	GET	`/clubs`	Lista todos lo
`clubsApi.get(id)`	GET	`/clubs/{id}`	Detalle de un
`clubsApi.create(data)`	POST	`/clubs`	Crea un club
`clubsApi.update(id, data)`	PATCH	`/clubs/{id}`	Edita nombre/desc
`clubsApi.delete(id)`	DELETE	`/clubs/{id}`	Elimina el clu
`clubsApi.join(id)`	POST	`/clubs/{id}/join`	Unirse o solic
`clubsApi.leave(id)`	DELETE	`/clubs/{id}/leave`	Abandonar el
`clubsApi.members(id)`	GET	`/clubs/{id}/members`	Lista de miem
`clubsApi.kickMember(clubId, userId)`	DELETE	`/clubs/{clubId}/members/{userId}`	Expulsar mien
`clubsApi.requests(id)`	GET	`/clubs/{id}/requests`	Solicitudes pe
`clubsApi.approveRequest(clubId, requestId)`	POST	`/clubs/{clubId}/requests/{requestId}/approve`	Aceptar solici
`clubsApi.rejectRequest(clubId, requestId)`	POST	`/clubs/{clubId}/requests/{requestId}/reject`	Rechazar solic
`clubsApi.setCurrentBook(clubId, externalId, from, until)`	PUT	`/clubs/{clubId}/current-book`	Establece el li

`clubsApi.removeCurrentBook(clubId)`	DELETE	`/clubs/{clubId}/current-book`	Quita el libro

Tipo `Club`**:** `id`, `name`, `description`, `visibility` (`public|private`), `memberCount`, `currentBook`, `userRole` (`admin|member|null`), `hasPendingRequest`, `owner`.

6.8 Módulo de chats — `api/chats.ts`

Función	Método	Endpoint	Descripción
`chatsApi.list(clubId)`	GET	`/clubs/{clubId}/chats`	Lista hilos de un club
`chatsApi.create(clubId, title)`	POST	`/clubs/{clubId}/chats`	Crea un hilo
`chatsApi.delete(clubId, chatId)`	DELETE	`/clubs/{clubId}/chats/{chatId}`	Elimina un hilo
`chatsApi.messages(clubId, chatId, page)`	GET	`/clubs/{clubId}/chats/{chatId}/messages`	Mensajes paginados del hilo
`chatsApi.sendMessage(clubId, chatId, content)`	POST	`/clubs/{clubId}/chats/{chatId}/messages`	Envía un mensaje
`chatsApi.deleteMessage(clubId, chatId, msgId)`	DELETE	`/clubs/{clubId}/chats/{chatId}/messages/{msgId}`	Elimina un mensaje

Tipo `Chat`**:** `id`, `title`, `isOpen`, `createdAt`, `closedAt`, `messageCount`, `createdBy`.
Tipo `ChatMessage`**:** `id`, `content`, `createdAt`, `user`.

6.9 Módulo de publicaciones — `api/posts.ts`

Función	Método	Endpoint	Descripción
`postsApi.feed()`	GET	`/posts`	Feed de usuarios seguidos
`postsApi.byUser(userId)`	GET	`/users/{userId}/posts`	Posts de un usuario concreto
`postsApi.create(image, description)`	POST	`/posts`	Publica con imagen (FormData)
`postsApi.delete(postId)`	DELETE	`/posts/{postId}`	Elimina la publicación
`postsApi.like(postId)`	POST	`/posts/{postId}/like`	Da/quita like (toggle)
`postsApi.comments(postId)`	GET	`/posts/{postId}/comments`	Lista comentarios
`postsApi.addComment(postId, content)`	POST	`/posts/{postId}/comments`	Añade un comentario

`postsApi.deleteComment(postId, commentId)`	DELETE	`/posts/{postId}/comments/{commentId}`	Elimina un comentario

Tipo `Post`: `id`, `imagePath`, `description`, `createdAt`, `likes`, `liked`, `commentCount`, `user`.

6.10 Módulo de perfil — `api/profile.ts`

Función	Método	Endpoint	Descripción
`profileApi.get()`	GET	`/profile`	Datos completos del perfil propio
`profileApi.update(data)`	PUT	`/profile`	Actualiza nombre visible y bio
`profileApi.uploadAvatar(file)`	POST	`/profile/avatar`	Sube nueva foto de perfil
`profileApi.updatePrivacy(data)`	PUT	`/profile/privacy`	Cambia configuración de privacidad
`profileApi.changePassword(current, new)`	PUT	`/profile/password`	Cambia la contraseña

Tipo `ProfileData`: `id`, `email`, `displayName`, `bio`, `avatar`, `shelvesPublic`, `clubsPublic`, `isPrivate`, `followers`, `following`.

6.11 Módulo de usuarios — `api/users.ts`

Función	Método	Endpoint	Descripción
`usersApi.search(q)`	GET	`/users/search?q=...`	Busca usuarios por nombre

Tipo `UserResult`: `id`, `displayName`, `avatar`, `bio`, `followers`, `followStatus` (`none|pending|accepted`), `isMe`.

6.12 Módulo de administración — `api/admin.ts`

Función	Método	Endpoint	Descripción
`adminApi.stats()`	GET	`/admin/stats`	Totales de usuarios, clubs y posts
`adminApi.users()`	GET	`/admin/users`	Lista completa de usuarios
`adminApi.setRole(id, isAdmin)`	PATCH	`/admin/users/{id}/role`	Da o quita rol de administrador
`adminApi.deleteUser(id)`	DELETE	`/admin/users/{id}`	Elimina una cuenta de usuario
`adminApi.clubs()`	GET	`/admin/clubs`	Lista completa de clubs
`adminApi.deleteClub(id)`	DELETE	`/admin/clubs/{id}`	Elimina un club
`adminApi.posts()`	GET	`/admin/posts`	Lista completa de publicaciones
`adminApi.deletePost(id)`	DELETE	`/admin/posts/{id}`	Elimina una publicación

Tipo `AdminStats`: `users`, `clubs`, `posts` (enteros con totales).

---

7. Componentes reutilizables

7.1 Navbar

Archivo: `components/Navbar.tsx`
Barra de navegación global, fija en la parte superior con `position: sticky` y efecto `backdrop-filter: blur(20px)`. Se detecta el scroll de la página y se añade una sombra al superar los 10px de desplazamiento.

Comportamiento por estado de autenticación:

Estado	Enlace visibles
No autenticado	Inicio, Libros, Clubs, Usuarios + botones Login/Registro
Autenticado	Todo lo anterior + Mis Estanterías
Administrador	Todo lo anterior + Administración

Sistema de notificaciones:

- Icono de campana que sondea `GET /api/notifications` cada **60 segundos**.
- Muestra un badge rojo con el número de notificaciones no leídas.
- Al hacer clic se abre un desplegable con el historial de las últimas 72 horas.
- Tipos de notificación y sus acciones inline:

Tipo	Descripción	Acción inline
`follow`	Alguien te sigue	Ninguna
`follow_request`	Alguien solicita seguirte	Aceptar / Rechazar
`follow_accepted`	Aceptaron tu solicitud de seguimiento	Ninguna
`club_request`	Solicitud de ingreso a tu club	Aceptar / Rechazar (desde la notif.)
`club_approved`	Tu solicitud de club fue aceptada	Ninguna
`club_rejected`	Tu solicitud de club fue rechazada	Ninguna
`like`	Alguien dio like a tu post	Ninguna
`comment`	Alguien comentó tu post	Ninguna

Menú de usuario: Avatar circular (DiceBear Initials como fallback), nombre visible y botón de cerrar sesión. Se cierra al hacer clic fuera del área.

Versión móvil (< 768px): El menú principal se oculta y aparece un botón hamburguesa. Al activarse, el overlay ocupa toda la pantalla y el scroll del body se bloquea. El menú hamburguesa tiene `aria-expanded` para accesibilidad.

Función auxiliar `timeAgo(date)`: Formatea la antigüedad de una notificación en español ("hace 5 min", "hace 2 h", "hace 3 d").

7.2 Footer

Archivo: `components/Footer.tsx`
Pie de página estático con tres columnas: descripción de la plataforma, enlaces de navegación y enlaces de redes sociales (Instagram y Twitter/X, con URLs de ejemplo). Adaptado a una sola columna en móvil.

7.3 PrivateRoute

Archivo: `components/PrivateRoute.tsx`

```
interface Props { children: React.ReactNode }
```

Mientras `loading` es `true`, muestra un spinner centrado para evitar parpadeos de redirección durante la verificación inicial de sesión. Una vez resuelta la verificación, redirige o renderiza según el estado de autenticación.

7.4 Toast / useToast

Archivo: `components/Toast.tsx`
Sistema de notificaciones no bloqueantes basado en una cola gestionada con `useReducer`.

- `<ToastProvider>` envuelve la aplicación y expone el contexto.
- `useToast()` devuelve `{ toast }` donde `toast(message, type)` añade una notificación a la cola.
- Cada toast tiene un identificador autoincremental, un tipo (`success | error | info`) y se elimina automáticamente a los 3 500 ms.
- El contenedor de toasts está posicionado como `position: fixed` en la esquina inferior derecha con `aria-live="polite"`.

7.5 PostCard

Archivo: `components/PostCard.tsx`

```
interface Props {
  post: Post
  meId: number | null      // ID del usuario autenticado
  onDelete?: (id: number) => void
  hideAuthor?: boolean     // Oculta la fila de autor (útil en perfil propio)
}
```

Funcionalidades:

- Muestra imagen del post (con `loading="lazy"`), avatar y nombre del autor, descripción y timestamp.
- Botón de **like**: toggle con animación; deshabilitado si no hay sesión. Llama a `postsApi.like()`.
- Botón de **eliminar** (solo visible si `meId === post.user.id`): abre confirmación nativa y llama a `onDelete`.
- Sección de **comentarios** colapsable. Al expandir, carga los comentarios (`postsApi.comments()`).
- Formulario de **nuevo comentario** (solo autenticados): textarea + botón de envío. Llama a `postsApi.addComment()`.
- Eliminar comentario: disponible para el autor del post y para el autor del comentario.

7.6 Spinner

Archivo: `components/Spinner.tsx`

```
interface Props { size?: number }  // px; por defecto 20
```

Renderiza `<span className="spinner" aria-label="Cargando" style={{ width, height }} />`. La animación `spin` está definida en `tokens.css`. Se usa inline dentro de botones para indicar que una operación asíncrona está en curso, sustituyendo el texto del botón sin cambiar su tamaño.

---

8. Páginas y flujos de usuario

8.1 HomePage — `/`

Propósito: Punto de entrada de la aplicación. Presenta la propuesta de valor y, si el usuario está autenticado, muestra el feed social personalizado.

Secciones:

1. **Hero**: texto eyebrow "Plataforma de lectura", título animado con `font-display`, descripción y tres estadísticas (10M+ libros, 3 modos de búsqueda, clubs ilimitados). Las llamadas a la acción varían según el estado de sesión:
   - Sin sesión: botones a `/register` y `/clubs`.
   - Con sesión: botones a `/clubs` y `/shelves`.
2. **Feed social** (solo autenticados): rejilla de componentes `<PostCard>` cargada desde `GET /api/posts`. Si el usuario no sigue a nadie todavía, se muestra un estado vacío con enlace a `/users` para descubrir lectores.
3. **Sección de funcionalidades**: tres tarjetas estáticas que describen las áreas principales de la plataforma (búsqueda de libros, estanterías personales, clubs de lectura).

Flujo de carga del feed:

1. `useEffect` se activa en cuanto `user` está disponible en `AuthContext`.
2. Llama a `postsApi.feed()`.
3. Durante la carga se muestra `<Spinner size={32} />` centrado.
4. Los errores de la petición se capturan silenciosamente para no romper la página principal ante un fallo secundario.

8.2 LoginPage — `/login`

Propósito: Autenticar al usuario con correo electrónico y contraseña.

Campos: Email (`type="email"`, `autoComplete="email"`) y contraseña (`autoComplete="current-password"`).

Flujo:

1. Al enviar el formulario se llama a `useAuth().login(email, password)`.
2. Internamente, se hace `POST /api/login` con las credenciales en JSON.
3. Si el servidor confirma la sesión, se redirige a `location.state.from` (URL de origen si el usuario fue enviado desde una ruta protegida) o a `/` en caso contrario.

Estados:

- Mientras se procesa: botón muestra `<Spinner>` y está deshabilitado (previene envíos duplicados).
- Error (401 credenciales incorrectas, error de red): alerta roja con el mensaje del servidor o "Credenciales incorrectas" por defecto.

8.3 RegisterPage — `/register`

Propósito: Crear una nueva cuenta de usuario.

Campos: Nombre visible, email, contraseña, confirmación de contraseña.

Validación en cliente (antes de cualquier petición):

- Contraseña con al menos 6 caracteres → "La contraseña debe tener al menos 6 caracteres".
- Las dos contraseñas deben coincidir → "Las contraseñas no coinciden".

Flujo:

1. Validación en cliente superada → `POST /api/auth/register`.
2. Tras registro exitoso, se llama automáticamente a `login(email, password)` del `AuthContext`.
3. Redirige a `/`.

Estados de error: alerta roja con el mensaje del servidor (p. ej. "El email ya está registrado" en caso de HTTP 409).

8.4 BooksPage — `/books`

Propósito: Buscar libros en el catálogo de Google Books e importarlos a estanterías personales.

Elementos:

- Selector de modo de búsqueda: texto libre / por título / por autor.
- Chips de sugerencias predefinidas (Harry Potter, 1984, El Quijote, etc.) que rellenan el campo automáticamente.
- Barra de búsqueda con botón "Buscar".
- Rejilla de tarjetas de libro con portada, título, autores, valoración media de Google Books y número de reseñas.
- Paginador con "Anterior" / "Siguiente" y contador de página.

Flujo de búsqueda:

1. El usuario escribe y pulsa "Buscar" o selecciona una sugerencia.
2. `GET /api/books/search?q=...&page=1&limit=12` (o `?title=...` / `?author=...` según el modo).
3. Los resultados se muestran con portada o con el icono `<BookOpen>` si no hay imagen disponible.
4. Cambiar de página mantiene el término de búsqueda y el modo seleccionado.

ShelfDrawer — subcomponente interno:
Panel lateral que se abre al pulsar "+ Estantería" sobre cualquier libro. Permite:

- Ver la lista de estanterías propias y añadir el libro a cualquiera de ellas (`POST /api/shelves/{id}/books`).
- Crear una nueva estantería sin cerrar el drawer (`POST /api/shelves`).
- Marcar el libro como "Estoy leyendo" (`POST /api/reading-progress`).

Una vez añadido el libro a una estantería, el botón correspondiente cambia a estado confirmado con `<CheckCircle>` verde y queda deshabilitado. El drawer se cierra al pulsar el botón X o haciendo clic en el overlay.

Estados:

- Sin resultados: "No se encontraron resultados. Prueba con otros términos."
- Error de red: alerta roja sobre la rejilla.
- Usuario no autenticado: el botón "+ Estantería" no se renderiza.

8.5 BookDetailPage — `/books/:externalId`

Propósito: Ver la información completa de un libro, gestionar el progreso de lectura y leer y escribir reseñas de la comunidad.

Secciones:

1. **Cabecera**: portada grande, título, autores, editorial, año de publicación, ISBN, número de páginas, idioma, categorías y enlace a Google Books. La imagen de portada se usa como fondo difuminado (blur) detrás de la cabecera.
2. **Valoración global**: puntuación media de la comunidad con distribución visual estilo Amazon (barras proporcionales para cada puntuación 1-5) y total de valoraciones.
3. **ShelfDrawer**: mismo componente que en `BooksPage`.
4. **Formulario de reseña** (autenticados): selector interactivo de estrellas con efecto hover, textarea de texto libre y botón de guardar/actualizar.
5. **Lista de reseñas de la comunidad**: avatar DiceBear, nombre del usuario, fecha, puntuación y texto. El usuario puede editar (precarga sus valores) o eliminar su propia reseña.

Flujo de carga: Al montar el componente se hacen tres peticiones en paralelo: `GET /api/books/{externalId}`, `GET /api/books/{externalId}/reviews` (incluye stats y la reseña propia del usuario).

Flujo de reseña:

1. Guardar: `POST /api/books/{externalId}/reviews` → la reseña aparece en la lista sin recargar.
2. Editar: misma petición con los valores actualizados.
3. Eliminar: confirmación inline → `DELETE /api/books/{externalId}/reviews`.

Estados de error: libro no encontrado → mensaje con enlace "Volver a libros"; error al publicar reseña → mensaje inline rojo bajo el formulario.

8.6 ClubsPage — `/clubs`

Propósito: Explorar todos los clubs de lectura, crear uno nuevo y gestionar la adhesión.

Elementos:

- Botón "Nuevo club" (solo autenticados) que expande un formulario de creación.
- Campo de búsqueda para filtrar clubs por nombre en tiempo real (filtrado local, sin petición al servidor).
- Rejilla de `<ClubCard>` con nombre, badge de visibilidad, badge de rol del usuario, descripción, número de miembros y portada del libro del mes.

Botón de acción en cada ClubCard:

Estado del usuario	Texto del botón	Acción
Sin rol, club público	"Unirse"	`POST /api/clubs/{id}/join`
Sin rol, club privado	"Solicitar unirse"	`POST /api/clubs/{id}/join`
Solicitud pendiente	"Solicitud enviada"	Deshabilitado (icono de reloj)
Miembro	"Abandonar"	Confirmación + `DELETE /api/clubs/{id}/leave`

Admin	Sin botón de acción	Accede a la gestión desde "Ver club"

Crear club:

1. Nombre (obligatorio), descripción y visibilidad (público/privado).
2. `POST /api/clubs` → el nuevo club aparece al inicio de la lista.

8.7 ClubDetailPage — `/clubs/:id`

Propósito: Centro de actividad del club. Concentra chats, gestión de miembros y solicitudes de adhesión.

Cabecera: Nombre, badge de visibilidad, descripción y, si hay libro del mes, su portada con el período de lectura. Los administradores ven además botones de edición del club y de cambio del libro del mes.

Pestaña Chats:

- Lista de hilos de conversación. Al seleccionar uno se carga el historial de mensajes.
- Cada mensaje propio aparece alineado a la derecha; los ajenos, a la izquierda.
- Formulario de envío al pie del hilo. Los mensajes nuevos aparecen al final sin recargar.
- Los administradores pueden crear nuevos hilos y eliminar cualquier mensaje.

Pestaña Miembros:

- Lista de todos los miembros con avatar, nombre y fecha de adhesión.
- Los administradores ven un botón de expulsión en cada miembro (con confirmación nativa).

Pestaña Solicitudes (solo visible para administradores en clubs privados):

- Lista de solicitudes pendientes de ingreso con botones "Aceptar" y "Rechazar".

BookMonthModal:

1. El admin busca un libro por texto libre.
2. Selecciona el resultado deseado (resaltado con borde de color al hacer clic).
3. Introduce fecha de inicio y fecha de fin (validación: fin > inicio).
4. `PUT /api/clubs/{id}/current-book` → la portada aparece en la cabecera del club.

8.8 ShelvesPage — `/shelves` *(ruta protegida)*

Propósito: Gestionar estanterías personales de libros y hacer seguimiento del progreso lector.

Layout: Sidebar izquierdo con la lista de estanterías y formulario de creación; panel derecho con los libros de la estantería seleccionada.

ReadingTracker (panel colapsable en la parte superior):

- Solo visible si hay al menos un libro en seguimiento activo.
- Cada `ReadingCard` muestra portada, título, barra de progreso animada y controles.
- **Modo porcentaje (%)**: input numérico 0-100.
- **Modo páginas**: input de página actual y total de páginas. Si el libro tiene `pageCount`, el total se precarga automáticamente.
- Guardar: `PATCH /api/reading-progress/{id}`.
- Eliminar: confirmación inline "¿Quitar?" → `DELETE /api/reading-progress/{id}` → toast de confirmación.

Operaciones de estanterías:

Acción	Endpoint	Retroalimentación
Crear	`POST /api/shelves`	Toast "Estantería [nombre] creada"
Eliminar	`DELETE /api/shelves/{id}`	Toast "Estantería [nombre] eliminada"
Mover libro	`POST /api/shelves/{src}/books/{id}/move`	Toast "[título] movido a [destino]"
Quitar libro	`DELETE /api/shelves/{id}/books/{bookId}`	Toast "[título] quitado de la estantería"

Estados vacíos:

- Sin estanterías: instrucción de crear la primera desde el formulario lateral.
- Estantería vacía: estado vacío con botón "Buscar libros" que navega a `/books`.

8.9 ProfilePage — `/profile` *(ruta protegida)*

Propósito: Configuración de cuenta, gestión de publicaciones y administración de relaciones sociales.

Secciones:

1. **Sidebar de perfil**: avatar, nombre visible, email, bio, contadores de seguidores y siguiendo (clicables para abrir el modal correspondiente).

2. **Mis publicaciones**: rejilla de posts propios (`postsApi.byUser(meId)`) con el componente `<PostCard hideAuthor>`. Formulario para crear nueva publicación (ver flujo abajo).

3. **Información personal**: campos de nombre visible y bio → `PUT /api/profile`. Confirmación con alerta verde que desaparece en 3 segundos; errores con alerta roja.

4. **Foto de perfil**: zona de clic que abre el selector de archivos (`accept="image/*"`). La imagen seleccionada se previsualiza en tiempo real con `FileReader` antes de confirmar la subida (`POST /api/profile/avatar` como `multipart/form-data`). Sin avatar personalizado, se genera automáticamente con DiceBear Initials.

5. **Privacidad**: tres toggles con guardado automático (sin botón de guardar):

   - Perfil privado: las solicitudes de seguimiento requieren aprobación.
   - Estanterías públicas: visitantes pueden ver las estanterías.
   - Clubs públicos: visitantes pueden ver los clubs.

   Cada toggle llama a `PUT /api/profile/privacy` al cambiar. Si la llamada falla, el toggle vuelve automáticamente al valor anterior.

6. **Cambiar contraseña**: validación en cliente (nueva ≥ 6 caracteres, confirmación coincide) → `PUT /api/profile/password`.

7. **Sesión**: muestra el email actual + botón "Cerrar sesión" → `POST /api/logout` + redirige a `/`.

Crear publicación:

1. "Nueva publicación" expande el formulario.
2. Click en la zona de dropzone abre el selector de archivo.
3. La imagen se previsualiza en tiempo real.
4. El botón "Publicar" está deshabilitado hasta seleccionar imagen.

5. `POST /api/posts` como `multipart/form-data` → la publicación aparece al inicio de la rejilla.

Modal de seguidores / siguiendo:

- Abre al pulsar el contador correspondiente.
- Muestra la lista de usuarios con avatar y nombre.
- En "Seguidores": botón X para eliminar a cada seguidor.

8.10 PublicProfilePage — `/users/:id`

Propósito: Ver el perfil público de otro usuario y gestionar el seguimiento.

Contenido (condicionado por la configuración de privacidad del usuario visitado):

- Avatar, nombre, bio, contadores de seguidores y siguiendo.
- Publicaciones del usuario (siempre visibles si no es cuenta privada o si se le sigue).
- Estanterías (solo si `shelvesPublic = true`).
- Clubs (solo si `clubsPublic = true`).

Lógica del botón de seguimiento:

Estado	Texto del botón	Acción
No seguido, público	"Seguir"	`POST /api/users/{id}/follow` → "Siguiendo"
No seguido, privado	"Seguir"	Crea solicitud pendiente → "Pendiente"
Pendiente	"Pendiente"	Deshabilitado
Siguiendo	"Siguiendo"	`DELETE /api/users/{id}/follow` → "Seguir"
Propio perfil	No aparece	Enlace a `/profile`

8.11 UsersPage — `/users`

Propósito: Descubrir otros lectores por nombre y gestionar el seguimiento.

Flujo de búsqueda:

1. El usuario escribe en el campo (mínimo 2 caracteres para activar la búsqueda).
2. Debounce de **350 ms** antes de llamar a `GET /api/users/search?q=...`. Esto evita peticiones excesivas mientras el usuario sigue escribiendo.
3. Spinner dentro del campo de búsqueda durante la petición.

Tarjeta de resultado:

- Avatar, nombre, bio y contador de seguidores.
- Botón de seguimiento dinámico idéntico al de `PublicProfilePage`.
- Si el resultado es el propio usuario: enlace "Mi perfil" → `/profile`.

Estados:

- < 2 caracteres: instrucción inicial de búsqueda.
- Sin resultados: "Nadie coincide con [término]."

8.12 AdminPage — `/admin`

Propósito: Panel de gestión de la plataforma para administradores.

Control de acceso: Verifica `user?.roles?.includes('ROLE_ADMIN')` al montar. Si no se cumple, redirige inmediatamente a `/` con `useNavigate`.

Pestañas:

1. **Estadísticas**: tres tarjetas con gradiente (violeta, cian, rosa) mostrando los totales actuales de usuarios, clubs y publicaciones. Fuente: `GET /api/admin/stats`.
2. **Usuarios**: tabla con email, nombre visible, avatar y acciones.
   - Campo de búsqueda para filtrar por nombre o email (local, sin petición).
   - "Dar admin" / "Quitar admin": confirmación nativa → `PATCH /api/admin/users/{id}/role`.
   - "Eliminar": confirmación nativa → `DELETE /api/admin/users/{id}`.
3. **Clubs**: tabla con nombre del club, descripción, número de miembros y propietario.
   - "Eliminar": confirmación nativa → `DELETE /api/admin/clubs/{id}`.
4. **Publicaciones**: cuadrícula de todas las publicaciones con imagen en miniatura, autor y fecha.
   - "Eliminar": confirmación nativa → `DELETE /api/admin/posts/{id}`.

---

9. Patrones y convenciones

9.1 Gestión de errores en tres niveles

Toda la aplicación aplica de forma consistente tres mecanismos de retroalimentación de errores:

Nivel	Mecanismo	Cuándo se usa
1	Estado de carga (`loading`)	Durante cualquier operación asíncrona
2	Alerta inline (`alert-danger`)	Cuando falla la carga inicial de la página
3	Toast (`useToast`)	En operaciones iniciadas activamente por el usuario

El spinner de nivel 1 deshabilita el botón para evitar envíos duplicados. Las alertas de nivel 2 usan las variables CSS `--color-danger-*`. Los toasts de nivel 3 se autodismiss a los 3 500 ms.

9.2 Formularios controlados

Todos los formularios usan el patrón de **entrada controlada** de React: cada campo tiene un estado local (`useState`) enlazado con `value` y actualizado con `onChange`. No se usa ninguna biblioteca de gestión de formularios (Formik, React Hook Form, etc.).

9.3 Avatares con fallback DiceBear

Todos los avatares de usuario siguen este patrón:

```
const avatarUrl = user.avatar?.startsWith('http')
  ? user.avatar
  : user.avatar
    ? `/uploads/${user.avatar}`
```

```
    : `https://api.dicebear.com/7.x/initials/svg?seed=${encodeURIComponent(user.displayName
?? user.email)}&radius=50`
```

Si el usuario ha subido un avatar, se sirve desde `/uploads/`. Si no, DiceBear genera automáticamente un SVG con las iniciales del nombre.

9.4 Imágenes con carga diferida

Todas las portadas de libros y las imágenes de posts usan `loading="lazy"` para diferir la carga hasta que el elemento es próximo al viewport, reduciendo la carga inicial de la página.

9.5 Debounce en búsquedas

`UsersPage` aplica un debounce de 350 ms usando `useEffect` con limpieza del timeout:

```
useEffect(() => {
  if (query.length < 2) return
  const timer = setTimeout(() => fetchUsers(query), 350)
  return () => clearTimeout(timer)  // limpia si el usuario sigue escribiendo
}, [query])
```

9.6 Cierre de modales con overlay

Todos los modales y drawers se cierran al hacer clic en el overlay (el fondo oscuro exterior):

```
<div className="modal-overlay" onClick={e => { if (e.target === e.currentTarget) onClose()
}}>
```

La verificación `e.target === e.currentTarget` garantiza que el modal no se cierre al hacer clic dentro del panel.

9.7 Paginación de búsqueda de libros

La paginación de `BooksPage` muestra 12 resultados por página. Los botones "Anterior" y "Siguiente" modifican el estado `page`, que se incluye en la siguiente petición de búsqueda. El total de páginas se calcula como `Math.ceil(total / 12)`.

9.8 Subida de archivos

Las subidas de archivos (avatar en `ProfilePage`, imagen en la creación de posts) siguen el mismo patrón:

1. `<input type="file" accept="image/*">` oculto, activado por clic en un área visual.
2. Previsualización inmediata con `FileReader.readAsDataURL()`.
3. Envío como `FormData` mediante `apiFormData()`.
4. El servidor responde con la URL del archivo almacenado, que se guarda en el estado del componente.

9.9 Internacionalización

La interfaz está completamente en español (es-ES). Los mensajes de error, placeholders, etiquetas, botones y textos de estado vacío están todos en castellano. El formateo de fechas usa `toLocaleDateString('es-ES', { ... })` para adaptarse a las convenciones locales.

---

10. Configuración y despliegue

10.1 Vite — `vite.config.ts`

```
export default defineConfig({
  plugins: [react()],
  build: {
    outDir: '../public/app',   // La build se deposita en public/app del backend
    emptyOutDir: true,
  },
  server: {
    port: 5173,
    proxy: {
      '/api':     { target: 'http://127.0.0.1:8000', changeOrigin: true },
      '/uploads': { target: 'http://127.0.0.1:8000', changeOrigin: true },
    },
  },
})
```

La opción `outDir: '../public/app'` dirige los archivos compilados al directorio `public/app/` del proyecto Symfony, que Nginx sirve como raíz de la SPA. De esta forma, el frontend compilado se integra directamente en el árbol de archivos del backend sin necesidad de un contenedor separado.

10.2 TypeScript — `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "skipLibCheck": true
  }
}
```

El modo `strict` activa todas las comprobaciones de TypeScript (nullability, tipos implícitos, etc.). `noUnusedLocals` y `noUnusedParameters` obligan a mantener el código limpio durante el desarrollo. `skipLibCheck` omite la verificación de tipos en `node_modules` para acelerar la compilación.

10.3 Scripts de npm

Script	Comando	Descripción
`npm run dev`	`vite`	Servidor de desarrollo en `http://localhost:5173` con HMR
`npm run build`	`tsc -b && vite build`	Verificación de tipos + compilación de producción

`npm run preview`	`vite preview`	Previsualización local de la build de producción

10.4 Dependencias

Producción:

Paquete	Versión	Función
`react`	^18.3.1	Framework de componentes
`react-dom`	^18.3.1	Renderizado al DOM
`react-router-dom`	^6.26.1	Enrutamiento SPA en el cliente
`lucide-react`	^1.7.0	Biblioteca de iconos SVG

Desarrollo:

Paquete	Versión	Función
`typescript`	^5.5.3	Compilador TypeScript
`vite`	^5.4.1	Bundler y servidor de desarrollo
`@vitejs/plugin-react`	^4.3.1	Soporte JSX con Fast Refresh
`@types/react`	^18.3.5	Tipos TypeScript para React
`@types/react-dom`	^18.3.0	Tipos TypeScript para React DOM